VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	PEIT41P Natural Language Processing Lab

Semester	B.E. Semester VII
Subject	Natural Language Processing Lab
Subject Professor In-charge	Dr. Rasika Ransing
Laboratory	Lab 11

Student Name	
Roll Number	

Experiment Number	3
Experiment Title	To implement Named Entity Recognition (NER) using deep learning, transformer-based models, and Large Language Models (LLMs) for identifying and classifying named entities in text.
Resources / Required	Hardware: I/O Devices (Monitor, Keyboard, Mouse, CPU, RAM, etc.) Software: Python
Objectives (Skill Set / Knowledge Tested / Imparted)	To implement and analyze Deep Learning, Transformer-based, and LLM techniques for accurate Named Entity Recognition (NER) from text.
Theory	**Named Entity Recognition (NER)** Named Entity Recognition is an important **Natural Language Processing (NLP)** task that identifies entities in a text and assigns them to predefined categories. Common entity categories include: **PERSON (PER):** Names of people **LOCATION (LOC):** Geographical locations **ORGANIZATION (ORG):** Companies, institutions, and organizations **DATE:** Dates and time expressions **MONEY:** Monetary values **GPE:** Countries, cities, states, etc. **EVENT:** Named events **Example:** "*Ratan Tata visited Mumbai for a business meeting.*" NER can identify:

Text	Entity
Ratan Tata	PERSON
Mumbai	LOCATION

NER as a Sequence Labelling Problem

NER is generally formulated as a **sequence labelling problem**, where each token in a sentence is assigned an entity label.

The **BIO (Beginning-Inside-Outside)** tagging scheme is commonly used:

B-XXX: Beginning of an entity

I-XXX: Inside an entity

O: Outside any entity

Example:

"*Elon Musk founded SpaceX.*"

Token	Label
Elon	B-PER
Musk	I-PER
founded	O
SpaceX	B-ORG
.	O

Traditional NER Approaches

Earlier NER systems relied on manually designed linguistic rules and statistical models such as:

Regular expressions

Rule-based systems

Hidden Markov Models (HMM)

Maximum Entropy Models

Conditional Random Fields (CRF)

These approaches often require extensive feature engineering and may have difficulty handling variations in language and unseen entities.

Deep Learning-Based NER

Deep learning techniques automatically learn useful features from training data, reducing the need for manual feature engineering.

Common architectures include:

a) Recurrent Neural Networks (RNN)

RNNs process tokens sequentially and maintain information from previously processed tokens.

b) Long Short-Term Memory (LSTM)
LSTMs are a type of RNN designed to capture long-term dependencies using memory cells and gates.
c) Bidirectional LSTM (BiLSTM)
BiLSTM processes the sentence in both forward and backward directions, allowing the model to use information from both preceding and succeeding words.
A common NER architecture is:
Word Embeddings → BiLSTM → CRF → Entity Labels
The CRF layer helps model dependencies between adjacent labels and improves sequence-level predictions.

Transformer-Based NER
Transformers have significantly improved NER performance by using **self-attention mechanisms** to understand relationships between words in a sentence.
Unlike RNNs, Transformers can process tokens in parallel and capture contextual relationships regardless of their distance in a sentence.
Popular transformer models used for NER include:
BERT
RoBERTa
DistilBERT
ALBERT
DeBERTa
XLM-RoBERTa
A typical transformer-based NER architecture is:
Input Text → Tokenization → Transformer Encoder → Classification Layer → Entity Labels

BERT for NER
BERT (Bidirectional Encoder Representations from Transformers) generates contextual representations of words by considering both the left and right context.
For NER, BERT can be fine-tuned by adding a classification layer on top of its output representations.
For example:
"*Apple opened a new store in Pune.*"
The model uses contextual information to classify:
Apple → **ORG**

Pune → **LOC**
BERT-based models are particularly effective because the meaning of a token can change depending on its context.

Large Language Models (LLMs) for NER
Large Language Models can also perform NER using their ability to understand and generate natural language.
LLMs can perform NER through:
Zero-shot learning – identifying entities without task-specific training.
Few-shot learning – providing a few labelled examples in the prompt.
Prompt-based NER – instructing the model to extract and classify entities.
Fine-tuning – training an LLM on a domain-specific NER dataset.
Example Prompt:
Identify all person names, organizations, and locations in the following sentence and return them with their entity types.
LLM-based NER can be useful when labelled training data is limited and when entities or domains are highly diverse.

Tokenization in NER
Before applying a deep learning or transformer model, the input text must be converted into tokens.
Transformer models commonly use **subword tokenization**, where an unknown or uncommon word can be divided into smaller subword units.
For example:
unhappiness → un + happiness
Special care is required when mapping entity labels from words to multiple subword tokens.

NER Pipeline
The general NER process can be represented as:

- **Input Text**

↓

Text Preprocessing

↓

Tokenization

↓

Pre-trained/Fine-tuned Model

↓

Contextual Representation

↓

Entity Classification

↓

BIO/Entity Labels

↓

Extracted Named Entities

Evaluation Metrics

NER models are commonly evaluated using **Precision, Recall, and F1-score**.

Precision:

$Precision = \frac{TP}{TP+FP}$

It measures how many predicted entities are actually correct.

Recall:

$Recall = \frac{TP}{TP+FN}$

It measures how many actual entities were correctly identified.

F1-Score:

$F1 = 2 \times \frac{Precision \times Recall}{Precision+Recall}$

The **F1-score** is widely used as the primary evaluation metric for NER because it considers both precision and recall.

Applications of NER

NER is widely used in:

- Information extraction
- Question answering
- Search engines
- Text summarization
- Chatbots and virtual assistants
- News analysis
- Medical information extraction
- Legal document processing
- Social media analysis
- Recommendation systems
- Knowledge graph construction

Advantages of Transformer/LLM-Based NER

- Captures contextual meaning effectively.

	• Requires less manual feature engineering. • Handles complex linguistic patterns. • Can be adapted to different domains. • Multilingual transformer models can support multiple languages. • LLMs can perform zero-shot and few-shot entity extraction. **Limitations** • Transformer models can require significant computational resources. • High-quality labelled datasets may be required for fine-tuning. • Domain-specific entities can be difficult to recognize. • LLM outputs may sometimes be inconsistent or incorrectly formatted. • Subword tokenization can make entity-label alignment challenging. • Large models can have higher inference costs.
Program	**English language** Deep Learning Transformers Fine-Tuning Transformers LLMs **Any Indian Regional language (Marathi/Hindi/Bengali, etc.)** Deep Learning Transformers Fine-Tuning Transformers LLMs
Output	**English language** Deep Learning Transformers

	Fine-Tuning Transformers LLMs **Any Indian Regional language (Marathi/Hindi/Bengali, etc.)** Deep Learning Transformers Fine-Tuning Transformers LLMs
Conclusion	